

Student Guide: Monitoring Pods and Nodes (Kubernetes)

Goal

Learn the **most-used kubectl commands** to monitor:

- **Pods** (status, restarts, logs, events, resources)
 - **Nodes** (health, capacity, conditions, resource usage)
-

1) Quick Cluster Health Checks

1.1 Check nodes (is the cluster healthy?)

`kubectl get nodes`

What you learn

- Node status: Ready means the node can run workloads.
- Quick view of node age and Kubernetes version.

Common statuses

- Ready  healthy
 - NotReady  kubelet/network/runtime issue
 - SchedulingDisabled  node is cordoned (no new pods scheduled)
-

1.2 Check all pods across namespaces

`kubectl get pods -A`

What you learn

- Overall workload status across the cluster (default + kube-system + your namespaces).
-

2) Monitoring Pods (Day-to-day Commands)

2.1 List pods in a namespace

`kubectl get pods -n <namespace>`

Example:

```
kubectl get pods -n default
```

Key columns

- READY = containers ready / total containers
 - STATUS = Running/Pending/CrashLoopBackOff/ImagePullBackOff/etc.
 - RESTARTS = frequent restarts means instability
-

2.2 Watch pod status live (like “tail”)

```
kubectl get pods -n <namespace> -w
```

Use when

- You just deployed and want to watch rollout behavior in real-time.

Stop with: Ctrl + C

2.3 See which node a pod is running on (wide output)

```
kubectl get pods -n <namespace> -o wide
```

What you learn

- Pod IP, Node name, and where it got scheduled.
-

2.4 Drill into a pod (most important troubleshooting command)

```
kubectl describe pod <pod-name> -n <namespace>
```

What you learn

- Container image, env vars, mounts
 - Readiness/liveness probe status
 - Node placement
 - **Events section** (often the real reason: image pull errors, OOMKilled, scheduling failures)
-

2.5 View pod logs

Single-container pod

```
kubectl logs <pod-name> -n <namespace>
```

Multi-container pod (specify container)

```
kubectl logs <pod-name> -n <namespace> -c <container-name>
```

Follow logs (stream)

```
kubectl logs -f <pod-name> -n <namespace>
```

Previous logs (after crash/restart)

```
kubectl logs <pod-name> -n <namespace> --previous
```

Use when

- Pod is in CrashLoopBackOff and you want logs from the last failed run.
-

2.6 Check events (why pods failed / scheduling issues)

Namespace events

```
kubectl get events -n <namespace> --sort-by=.metadata.creationTimestamp
```

All namespaces

```
kubectl get events -A --sort-by=.metadata.creationTimestamp
```

What you learn

- ImagePullBackOff, FailedScheduling, BackOff, probes failing, etc.
-

2.7 Pod resource usage (CPU/RAM)

```
kubectl top pod -n <namespace>
```

Prerequisite

- metrics-server must be installed and working.

Useful options:

```
kubectl top pod -n <namespace> --containers
```

(Shows per-container CPU/memory)

2.8 Filter problem pods quickly

Only not-running pods

```
kubectl get pods -n <namespace> --field-selector=status.phase!=Running
```

Find pods restarting a lot (manual scan)

```
kubectl get pods -n <namespace>
```

Look for high RESTARTS.

3) Monitoring Nodes

3.1 Node details (deep diagnostics)

```
kubectl describe node <node-name>
```

What you learn

- Conditions: Ready, MemoryPressure, DiskPressure, PIDPressure, NetworkUnavailable
 - Capacity vs allocatable resources
 - Which pods are running on the node
 - Events related to node health
-

3.2 Node resource usage (CPU/RAM)

```
kubectl top node
```

Use when

- Pods are slow / evicted / throttled, and you suspect node is under pressure.
-

3.3 List pods running on a specific node

```
kubectl get pods -A -o wide --field-selector spec.nodeName=<node-name>
```

What you learn

- Exactly what workloads are consuming the node.

3.4 Check node labels (useful for scheduling)

kubectl get node <node-name> --show-labels

Use when

- Pods use nodeSelector, affinity/anti-affinity, or taints/tolerations.

4) Common Pod Status Meanings (Quick Reference)

- **Pending:** scheduler can't place pod (resource shortage, taints, PVC not bound)
- **CrashLoopBackOff:** app starts → crashes → restarts repeatedly
- **ImagePullBackOff / ErrImagePull:** image name/tag wrong, registry auth, network issues
- **OOMKilled:** container killed due to memory limit
- **Running but not Ready:** readiness probe failing, app not healthy yet

5) Practical Monitoring Flow (What to do in real incidents)

1. **Check pod state**
2. kubectl get pods -n <ns> -o wide
3. **Describe the failing pod**
4. kubectl describe pod <pod> -n <ns>
5. **Check logs**
6. kubectl logs <pod> -n <ns> --previous
7. **Check namespace events**
8. kubectl get events -n <ns> --sort-by=.metadata.creationTimestamp
9. **Check node health + usage**
10. kubectl get nodes
11. kubectl top node
12. kubectl describe node <node>

